



Módulo: Programación de Servicios y Procesos

UT1: PROGRAMACIÓN MULTIPROCESO

Código **0490**

Nivel **Ciclo Formativo de Grado Superior**
Ciclo **Desarrollo de Aplicaciones**
Multiplataforma

Índice

Resultados de aprendizaje y criterios de evaluación	4
1 Introducción a la programación multiproceso.....	5
1.1 Programa, ejecutable, proceso y servicio	5
1.2 Multitarea: concurrencia y paralelismo	7
1.3 Programación concurrente: características y ámbitos	8
1.4 Programación paralela y distribuida	9
2 El proceso y el sistema operativo.....	10
2.1 Principales estados de un proceso y cambios de estado	10
2.2 Bloque de control de procesos (PCB) y colas de procesos.....	11
2.3 Planificación de procesos (cargador, planificadores y algoritmos).....	12
2.4 Cambio de contexto	14
3 Gestión de procesos con herramientas del sistema	14
3.1 Administrador de tareas.....	14
3.2 Comandos en Windows: tasklist / taskkill.....	15
3.3 Equivalentes en GNU/Linux.....	16
4 Hilos frente a procesos.....	16
4.1 Qué es un hilo.....	16
4.2 Cuándo conviene un proceso y cuándo un hilo	17
5 Creación y gestión de procesos desde Java	19
5.1 Creación de procesos	19
5.1.1 Las clases Process y ProcessBuilder.....	19
5.1.2 Argumentos, directorio de trabajo y entorno	20
5.1.3 Lanzamiento y ciclo de vida del proceso hijo	22
5.2 Comunicación entre procesos	23
5.2.1 Entrada, salida y error estándar	23
5.2.2 Los flujos del hijo vistos desde el padre	24
5.2.3 Redirecciones y encadenamiento de procesos	26
5.3 Sincronización entre procesos	27
5.3.1 Espera del proceso hijo: waitFor().....	28
5.3.2 Códigos de salida	28
5.3.3 Espera asíncrona: onExit()	29
5.3.4 Consulta y terminación: ProcessHandle y destroy().....	30

6	Aplicaciones multiproceso en paralelo	31
6.1	Reparto de una tarea entre varios subprocesos	31
6.2	Medición frente a la ejecución secuencial	33
7	Documentación y depuración	35
	Bibliografía.....	37

Resultados de aprendizaje y criterios de evaluación

RESULTADO DE APRENDIZAJE: RA1

RA1.- Desarrolla aplicaciones compuestas por varios procesos reconociendo y aplicando principios de programación paralela.

CRITERIOS DE EVALUACIÓN:

a)	Se han reconocido las características de la programación concurrente y sus ámbitos de aplicación.
b)	Se han identificado las diferencias entre programación paralela y programación distribuida, sus ventajas e inconvenientes.
c)	Se han analizado las características de los procesos y de su ejecución por el sistema operativo.
d)	Se han caracterizado los hilos de ejecución y descrito su relación con los procesos.
e)	Se han utilizado clases para programar aplicaciones que crean subprocesos.
f)	Se han utilizado mecanismos para compartir información con los subprocesos iniciados.
g)	Se han utilizado mecanismos para sincronizar y obtener el valor devuelto por los subprocesos iniciados.
h)	Se han desarrollado aplicaciones que gestionen y utilicen procesos para la ejecución de varias tareas en paralelo.
i)	Se han depurado y documentado las aplicaciones desarrolladas.

1 Introducción a la programación multiproceso

¿Cuántos programas está ejecutando tu ordenador ahora mismo? ¿Cómo puede ejecutar doscientos a la vez si solo tiene ocho núcleos? ¿Por qué Chrome abre un proceso por cada pestaña? En esta unidad de trabajo responderemos a estas preguntas y aprenderemos a escribir programas en Java que **lanzan otros programas, se comunican con ellos y controlan cuándo terminan**.

Antes necesitamos un vocabulario común y preciso.

1.1 Programa, ejecutable, proceso y servicio

Estos cuatro términos están relacionados, pero **no significan lo mismo**. Por encima de todos está el **sistema operativo (SO)**: el programa que actúa de intermediario entre el usuario, las aplicaciones y el hardware. Cuando queremos ejecutar un programa, se lo pedimos al sistema operativo, y es él quien lo carga en memoria y le reparte tiempo de procesador.

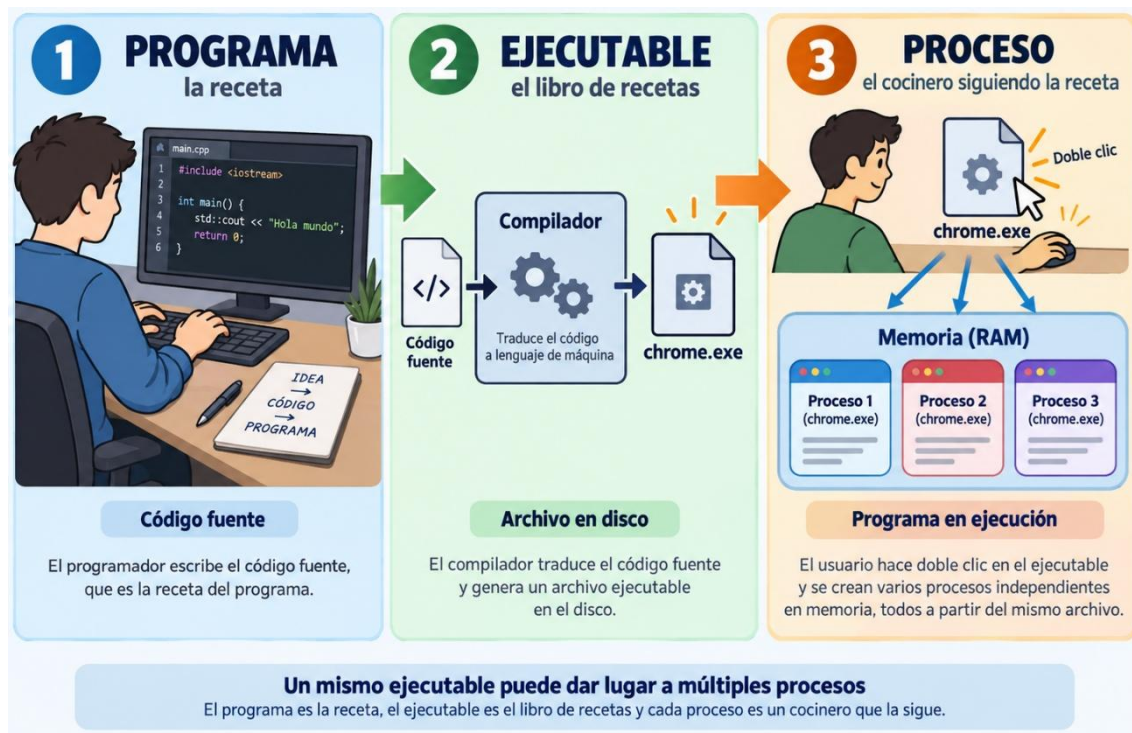


Imagen1.- de la idea a la ejecución (generada con IA)

Programa



Un **programa** es el conjunto de instrucciones que escribe el programador en un lenguaje de programación. Es el **código fuente**, por ejemplo el archivo `HolaMundo.java`. Es algo **estático**: por sí mismo no hace nada. Si lo comparamos con la cocina, el programa es **la receta**.

Ejecutable



El **compilador** traduce el código fuente a un formato que el sistema operativo sabe cargar y ejecutar. El resultado es el **ejecutable**: un archivo en disco listo para usarse, como [notepad.exe](#) o [chrome.exe](#). Sigue siendo estático: es **el libro de recetas**.

En Java, el compilador [javac](#) genera archivos [.class](#) con *bytecode*, y quien se ejecuta es la máquina virtual de Java, [java.exe](#). Por eso verás [java.exe](#) en la lista de procesos cuando ejecutes tus programas.

Proceso



Cuando el usuario abre un ejecutable, el sistema operativo lo carga en memoria y lo pone en marcha: el programa se convierte en un **proceso**. Es **el cocinero siguiendo la receta**. Además del código, cada proceso tiene **su propia memoria** y su propio estado de ejecución (lo veremos en el apartado 2).

Un mismo ejecutable puede dar lugar a varios procesos. Si abres [chrome.exe](#) tres veces, tienes tres procesos independientes: si uno falla, los otros siguen funcionando.



Idea clave

Proceso = programa en ejecución. Es la definición fundamental de esta unidad de trabajo.

Servicio

Un **servicio** es un proceso con tres características particulares:

1. **Se ejecuta en segundo plano**: no tiene ventana visible.
2. **No requiere interacción** del usuario para funcionar.
3. **Se inicia automáticamente** con el sistema operativo y permanece siempre activo.

Ejemplos: el **antivirus**, el **servicio de actualización de Windows** o un **servidor de base de datos**.



Imagen 2.- Servicios ejecutando en segundo plano (generada con IA)



Idea clave

En Windows puedes ver los servicios con la herramienta [services.msc](#) (Windows + R, escribe services.msc y pulsa Intro). En GNU/Linux los servicios se llaman **demonios** (*daemons*).

1.2 Multitarea: concurrencia y paralelismo

Mientras escribes un documento, tu ordenador reproduce música, ejecuta el antivirus y descarga un archivo. Hay más de cien procesos en marcha y solo unos pocos núcleos. ¿Cómo es posible? Gracias a la **multitarea: la capacidad de un ordenador de ejecutar varias tareas simultáneamente**.

Según el hardware disponible, distinguimos dos tipos:

1. **Multitarea real (paralelismo):** hay tantos núcleos como tareas, o más. **Cada tarea se ejecuta en su propio núcleo al mismo tiempo.**
2. **Multitarea simulada (concurrencia):** hay menos núcleos que tareas. El sistema operativo **las alterna** a gran velocidad, dando a cada una unos milisegundos de procesador. Así parece que todas se ejecutan a la vez, aunque en cada instante un núcleo solo ejecuta una.

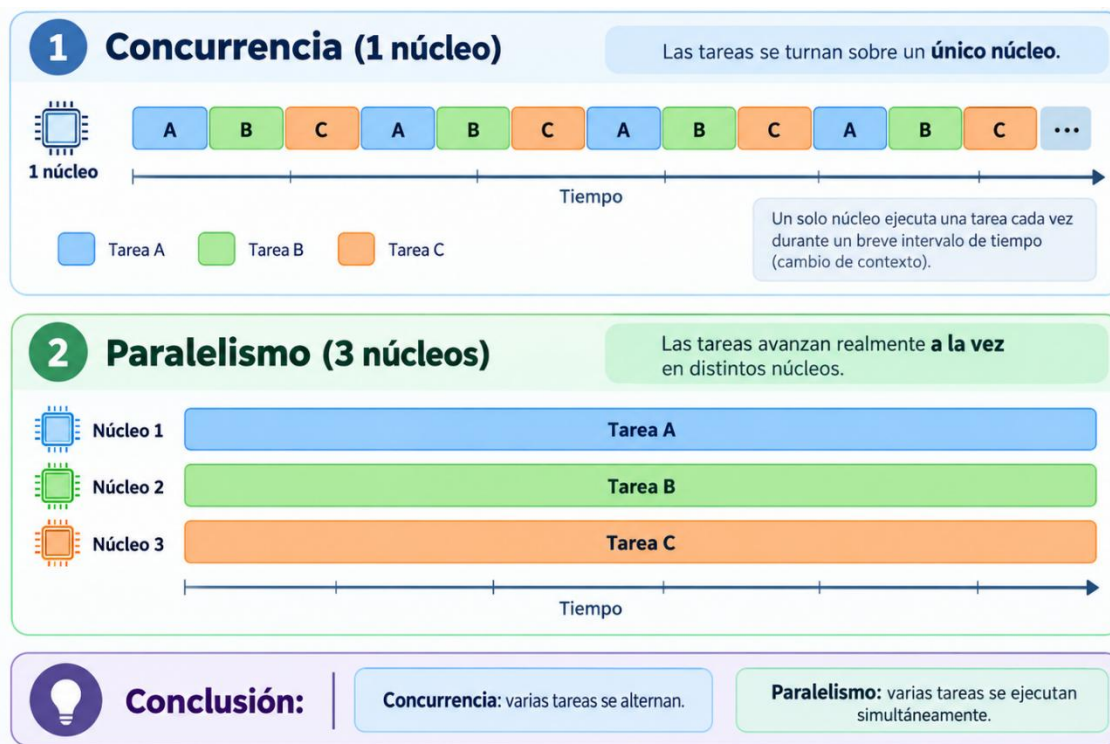


Imagen 3.- Concurrencia y paralelismo (generada con IA)

Un ordenador actual combina ambas: con ocho núcleos y doscientos procesos, ocho se ejecutan en paralelo y el resto se alternan en cada núcleo.



Idea clave

El hardware pone los núcleos; el sistema operativo decide quién entra y cuándo.

Toda ejecución paralela es concurrente. No toda ejecución concurrente es paralela.

CPU y sistema operativo

La multitarea necesita la colaboración de ambos:

- **El procesador (CPU)** aporta la capacidad física: los núcleos que ejecutan las instrucciones.
- **El sistema operativo** aporta la capacidad lógica: decide qué tarea ocupa cada núcleo y durante cuánto tiempo.

Por eso, **un procesador sencillo con un buen sistema operativo puede ofrecer multitarea**, y un procesador potente con un sistema operativo inadecuado no la aprovechará.

1.3 Programación concurrente: características y ámbitos

La **programación concurrente** consiste en diseñar **un programa formado por varias tareas que se ejecutan a la vez y colaboran para lograr un objetivo común**. Esas tareas pueden avanzar en paralelo o alternándose, según los núcleos disponibles.

¿Qué aporta?

1. **Aprovechamiento del procesador:** mientras una tarea espera al disco o a la red, otra usa el procesador. Sin concurrencia, ese tiempo se pierde.
2. **Capacidad de respuesta:** la interfaz sigue atendiendo al usuario mientras el programa realiza un trabajo pesado. Sin concurrencia, la aplicación queda bloqueada y Windows muestra el mensaje «No responde».
3. **Escalabilidad:** el trabajo puede repartirse entre más núcleos o más máquinas según sea necesario.



Idea clave

El precio: **coordinar varias tareas es mucho más difícil que programar una sola**. Hay que repartir el trabajo, pasar datos entre tareas y esperar a que terminen. El resto de esta unidad de trabajo trata de eso.

Ámbitos de aplicación

- **Servidores:** un servidor web atiende a miles de clientes a la vez. Sin concurrencia, atendería a uno y los demás esperarían.
- **Cálculo intensivo:** compilar un proyecto grande, renderizar vídeo, aplicar filtros a imágenes o ejecutar simulaciones científicas. El trabajo se divide y se reparte.

- **Aplicaciones interactivas:** videojuegos, editores o navegadores separan la interfaz del trabajo pesado.
- **El propio sistema operativo:** es el primer programa concurrente del ordenador. Sin él, nada de lo anterior sería posible.

1.4 Programación paralela y distribuida

Ambas dividen un problema en tareas que se ejecutan a la vez. La diferencia está en **dónde se ejecutan y cómo se comunican**:

- **Programación paralela:** las tareas se ejecutan en **varios núcleos de una misma máquina** y se comunican **a través de la memoria compartida**. Ejemplo: aplicar un filtro a una imagen dividiéndola en cuatro trozos, uno por núcleo.
- **Programación distribuida:** las tareas se ejecutan en **varias máquinas conectadas en red**. Cada máquina tiene su propia memoria, así que se comunican **enviándose mensajes**. Ejemplo: una tienda online cuyo catálogo, pagos y pedidos funcionan en servidores distintos.

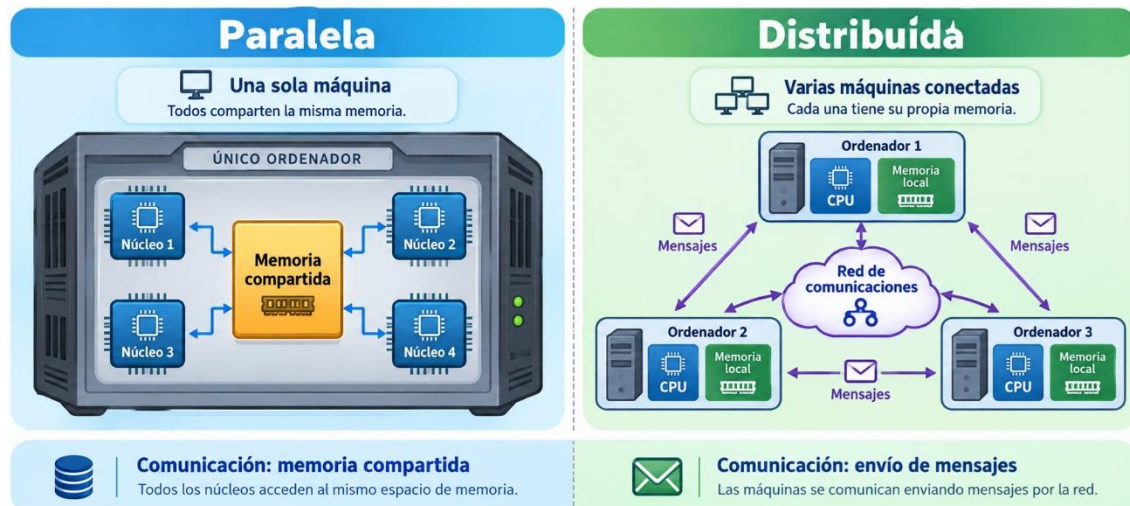


Imagen 4.- Comparativa programación paralela y distribuida (generada con IA)

	Paralela	Distribuida
Dónde se ejecuta	Varios núcleos, una máquina	Varias máquinas en red
Memoria	Compartida	Cada máquina tiene la suya
Cómo se comunican	Leyendo la misma memoria	Enviándose mensajes
Ventajas	Comunicación inmediata y programación más sencilla.	Crece añadiendo máquinas y sigue funcionando si cae una de ellas.
Inconvenientes	Limitada por los núcleos del equipo. Compartir memoria puede provocar errores cuando varias tareas acceden al mismo dato a la vez.	La red añade retardo, puede fallar a mitad de una operación y la depuración es mucho más compleja.

En esta unidad de trabajo nos centramos en el primer escenario: **varios procesos en una misma máquina**. La comunicación entre máquinas se estudia en las unidades de trabajo 3 y 4.

Queda una pregunta: ¿qué es exactamente un proceso para el sistema operativo y cómo lo gestiona? Lo vemos en el apartado 2.

2 El proceso y el sistema operativo

El sistema operativo controla todos los procesos: decide cuándo se ejecuta cada uno, dónde guarda su información y cómo reparte el procesador entre ellos. En este apartado veremos cómo lo hace.

2.1 Principales estados de un proceso y cambios de estado

Un proceso no está siempre ejecutándose; de hecho, casi nunca lo está. Desde que se crea hasta que termina, pasa por varios estados controlados por el sistema operativo:

1. **Nuevo:** el proceso se ha creado, pero todavía no está preparado para ejecutarse.
2. **Listo:** el proceso está preparado y **espera a que se le asigne el procesador**.
3. **En ejecución:** el proceso tiene el procesador y está ejecutando sus instrucciones.
4. **Bloqueado:** el proceso **espera un evento**, por ejemplo que termine una lectura del disco, y no puede avanzar hasta que ocurra.
5. **Suspendido:** el proceso se ha trasladado temporalmente a la memoria secundaria (disco) por falta de memoria principal.
6. **Terminado:** el proceso ha finalizado su ejecución y ha liberado sus recursos.

El paso de un estado a otro se llama **transición** o **cambio de estado**. Cada transición tiene una causa concreta:

Transición	Cambio	Causa
Admisión	Nuevo → Listo	El sistema operativo decide que hay espacio en memoria para el proceso.
Despacho	Listo → En ejecución	El planificador le asigna el procesador.
Expulsión	En ejecución → Listo	Se agota su tiempo asignado o llega un proceso más prioritario.
Bloqueo	En ejecución → Bloqueado	El propio proceso solicita una operación de entrada/salida.
Desbloqueo	Bloqueado → Listo	Termina la operación que estaba esperando.



Imagen 5.- Estados de un proceso y cambios de estado (generada con IA)



Atención

De Bloqueado nunca se pasa directamente a En ejecución. Cuando termina la operación que esperaba, el proceso vuelve a Listo y debe esperar de nuevo su turno.

2.2 Bloque de control de procesos (PCB) y colas de procesos

Bloque de control de procesos (PCB)

Cuando un proceso deja el procesador, el sistema operativo tiene que recordar exactamente cómo lo dejó para poder retomarlo después. Esa información se guarda en el **Bloque de Control de Proceso (PCB, *Process Control Block*)**. Existe un PCB por cada proceso activo, y contiene:

1. **Contador de programa:** indica por qué instrucción del código va el proceso.
2. **Imagen de memoria:** el espacio que ocupa el proceso en la memoria principal.
3. **Estado del procesador:** el valor de los registros de la CPU en el momento en que se interrumpió.
4. **Identificador (PID) y estado actual:** el número que identifica al proceso y el estado en que se encuentra.
5. **Recursos abiertos:** archivos, dispositivos y prioridad del proceso.



Idea clave

Sin PCB no habría multitarea: sería imposible retomar un proceso en el punto exacto en que se quedó.

Colas de procesos

El sistema operativo organiza los procesos en **colas**, según lo que estén esperando:

- **Cola de procesos listos:** contiene todos los procesos que podrían ejecutarse ya. De aquí elige el planificador el siguiente proceso que ocupará el procesador.
- **Colas de procesos bloqueados:** hay **una cola por cada dispositivo** (disco, red, teclado...). Cuando el dispositivo responde, el proceso sale de su cola y pasa a la de listos.

Cada cola es, en realidad, **una lista enlazada de PCB**. Por tanto, **cambiar un proceso de estado consiste en mover su PCB de una cola a otra**.

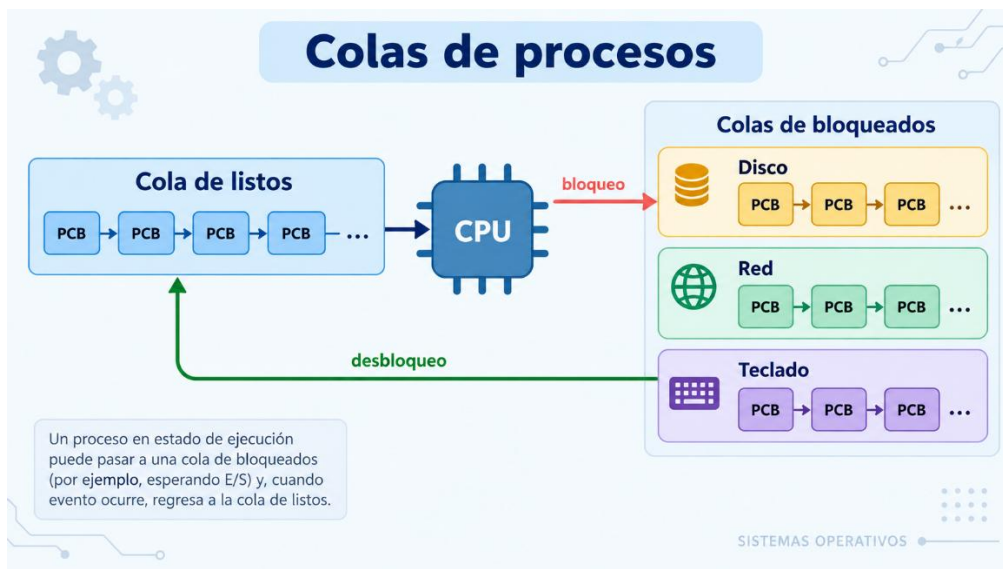


Imagen 6.- Colas de procesos (generada con IA)

2.3 Planificación de procesos (cargador, planificadores y algoritmos)

El cargador

Antes de que un proceso pueda competir por el procesador, alguien tiene que crearlo. De eso se encarga el **cargador**, que realiza cuatro tareas:

1. Reserva espacio en la memoria principal para el proceso.
2. Copia en ese espacio las instrucciones, los datos y la pila.
3. Crea el PCB con el PID y el estado inicial.
4. Coloca el proceso en la cola de listos.



A partir de ese momento, su trabajo ha terminado: **el cargador no decide cuándo se ejecuta el proceso; eso le corresponde al planificador.**

Idea clave

Todo lo que ocurre entre el doble clic sobre un icono y la aparición de la ventana es trabajo del cargador.

Los planificadores

El sistema operativo utiliza tres planificadores, que actúan en momentos distintos:

- **Planificador a largo plazo:** selecciona qué procesos se admiten en la memoria principal y los lleva al estado Listo.
- **Planificador a corto plazo:** **asigna el procesador a los procesos listos.** Es el que realiza los cambios entre Listo y En ejecución, y actúa muchas veces por segundo.
- **Planificador a medio plazo:** gestiona el **intercambio (swap)** entre memoria principal y secundaria, suspendiendo o reanudando procesos según la memoria disponible.



Imagen 7.- Estados de procesos y planificadores (generada con IA)

Los objetivos que persigue la planificación son tres:

1. **Equidad:** que ningún proceso se quede sin ejecutarse.
2. **Eficiencia:** que el procesador no esté parado.
3. **Tiempo de respuesta:** que el usuario no note la espera.

Algoritmos de planificación

Para decidir qué proceso se ejecuta y durante cuánto tiempo, el planificador sigue un **algoritmo de planificación**. Los más habituales son:

- **Round-Robin:** asigna a cada proceso un **quantum**, es decir, una porción fija de tiempo. Si el proceso no termina en ese tiempo, vuelve al final de la cola de listos. Es especialmente útil en sistemas interactivos.
- **Por prioridad:** cada proceso tiene una prioridad y el procesador se asigna primero a los de mayor prioridad. Es adecuado para procesos críticos que no deben esperar.
- **Múltiples colas:** combina los dos anteriores. Los procesos se organizan en colas según su prioridad y cada cola se gestiona con Round-Robin.



Atención

Un proceso de prioridad baja puede no ejecutarse nunca si no dejan de llegar procesos más prioritarios. Este problema se llama **inanición**.

2.4 Cambio de contexto

El **cambio de contexto** es lo que ocurre **cada vez que el procesador deja un proceso y pasa a ejecutar otro**. Se realiza en dos pasos:

1. **Guardado del estado del proceso saliente:** el sistema operativo almacena en su PCB el estado de la CPU, incluidos el **contador de programa** (la siguiente instrucción que debe ejecutar) y el **puntero de pila**.
2. **Restauración del estado del proceso entrante:** se cargan en la CPU los valores guardados en el PCB del nuevo proceso, que continúa exactamente donde se quedó.



Atención

El cambio de contexto no es gratuito: **mientras se realiza, el procesador no hace trabajo útil**. Con miles de cambios por segundo, el coste en rendimiento es apreciable.

Buena parte de ese coste se debe a que, al cambiar de proceso, también hay que cambiar de espacio de memoria. ¿Existe una alternativa más ligera dentro de un mismo proceso? Sí: los **hilos**, que veremos en el apartado 4.

3 Gestión de procesos con herramientas del sistema

Todo lo visto en el apartado 2 se puede observar en el propio ordenador. Windows ofrece una herramienta gráfica, el Administrador de tareas, y dos comandos, [tasklist](#) y [taskkill](#), para consultar y controlar los procesos.

3.1 Administrador de tareas

El **Administrador de tareas** es la herramienta gráfica de Windows para ver y controlar los procesos activos. Se abre con la combinación **Ctrl + Mayús + Esc**. Las dos pestañas que nos interesan son:

- **Procesos:** muestra los procesos agrupados por aplicación, con su consumo de CPU, memoria, disco y red en tiempo real. Al desplegar una aplicación se ven los procesos que la forman.
- **Detalles:** muestra **un proceso por fila**, con su **PID**, el usuario que lo lanzó y su estado. Es la vista que utilizaremos en esta unidad de trabajo.

Desde la pestaña Detalles, haciendo clic derecho sobre un proceso, podemos:

- **Finalizar la tarea**, es decir, terminar el proceso.
- **Establecer la prioridad del proceso.**
- **Finalizar el árbol de procesos**, que termina el proceso junto con todos los procesos hijos que ha creado.

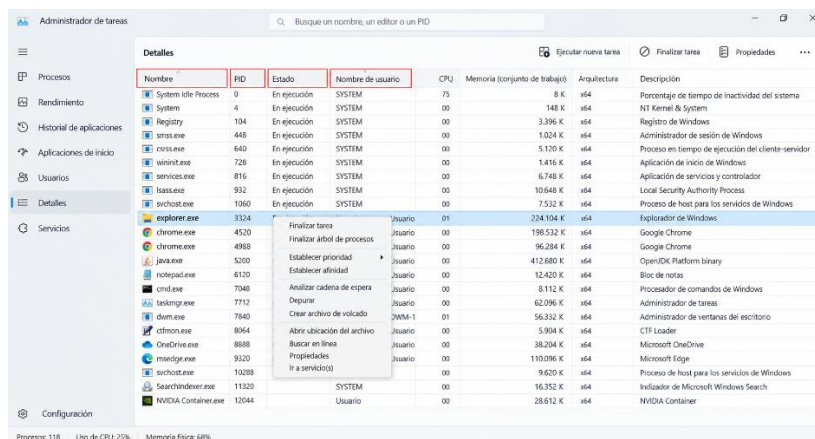


Imagen 8.- Administrador de tareas de Windows

3.2 Comandos en Windows: tasklist / taskkill

Los mismos datos se pueden consultar desde la consola (cmd) con dos comandos.

tasklist

Muestra la lista de procesos activos con su **nombre de imagen** (el ejecutable), su **PID** y la memoria que usan:

```
C:\> tasklist
```

Admite opciones para filtrar y cambiar el formato de la salida. Las más útiles son:

```
C:\> tasklist /FI "IMAGENAME eq notepad.exe" (procesos de notepad.exe)
C:\> tasklist /FI "PID eq 4820" (solo el proceso con PID 4820)
C:\> tasklist /FO CSV (salida en formato CSV)
```

taskkill

Termina un proceso a partir de su PID:

```
C:\> taskkill /PID 4820          (pide al proceso que se cierre)
C:\> taskkill /PID 4820 /F      (lo termina de forma forzada)
C:\> taskkill /PID 4820 /T /F   (termina el proceso y todos sus hijos)
```

Sin la opción **/F**, **taskkill** **solicita** al proceso que se cierre, y el proceso puede, por ejemplo, preguntar si se guardan los cambios. Con **/F** el proceso **se termina de inmediato**, sin posibilidad de guardar nada.



Idea clave

¿Por qué aprender estos comandos si ya existe el Administrador de tareas? Porque **desde Java se lanzan comandos, no ventanas**. En el apartado 5 los ejecutaremos desde nuestros programas.

3.3 Equivalentes en GNU/Linux

En clase trabajaremos con Windows, pero estos comandos los encontrarás en cualquier servidor GNU/Linux:

Windows	GNU/Linux	Qué hace
tasklist	ps aux	Lista todos los procesos del sistema.
—	pstree	Muestra los procesos como un árbol de padres e hijos.
taskkill /PID	kill PID	Termina un proceso por su identificador.
Establecer prioridad (Administrador de tareas)	nice -n	Lanza un proceso con una prioridad concreta.



Idea clave

Los conceptos son los mismos en ambos sistemas: PID, estado, prioridad y jerarquía padre-hijo. Solo cambia el nombre del comando.

4 Hilos frente a procesos

En el apartado 2 vimos que cambiar de un proceso a otro tiene un coste. Existe una unidad de ejecución más ligera, el hilo. En esta unidad de trabajo solo veremos **qué es un hilo y en qué se diferencia de un proceso**; su programación en Java se estudia en la unidad de trabajo 2.

4.1 Qué es un hilo

Un proceso puede realizar varias tareas a la vez dentro de sí mismo. Cada una de esas líneas de ejecución es un **hilo** (*thread*). **El hilo es la unidad básica de ejecución dentro**

de un proceso y también se conoce como **proceso ligero**. Todo proceso tiene al menos un hilo; en un programa Java, es el que ejecuta el método `main()`.

Los hilos de un mismo proceso **comparten**:

- **La memoria del proceso**: variables, objetos y datos.
- **Los archivos y conexiones abiertos** por el proceso.

Cada hilo tiene **para sí mismo**:

- **Su propio contador de programa**, que indica por qué instrucción va.
- **Su propia pila**, con sus variables locales y las llamadas a métodos en curso.

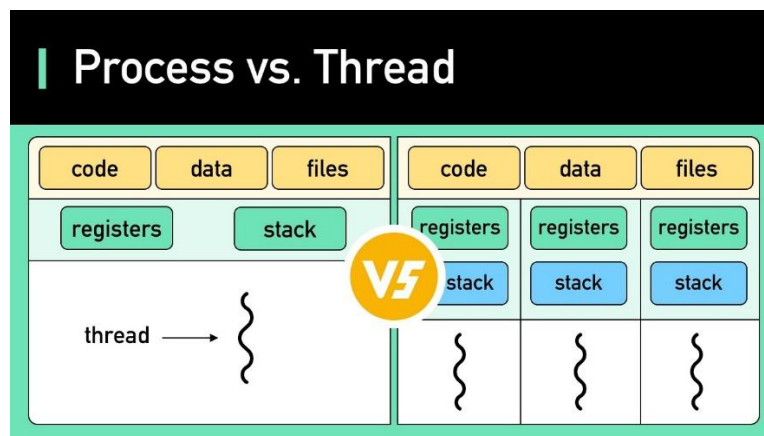


Imagen 9.- Comparativa proceso e hilo

[Enlace a vídeo descriptivo](#)

¿Por qué un hilo es más ligero que un proceso?

- **Crear un proceso es costoso**: hay que reservar un espacio de memoria nuevo, copiar el código y los datos, y construir un PCB completo.
- **Crear un hilo es barato**: el espacio de memoria ya existe; solo hay que asignarle una pila y un contador de programa.
- **Cambiar de hilo también es más barato**: el cambio de contexto entre dos hilos del mismo proceso no cambia el espacio de memoria, que es la parte más costosa del cambio de contexto visto en el apartado 2.4.

Sin embargo, **para el planificador, hilos y procesos se tratan igual**: reparte el tiempo de procesador entre hilos con los mismos estados y las mismas colas que entre procesos.



Idea clave

La ventaja de los hilos no está en cómo se planifican, sino en **lo poco que cuesta crearlos y alternarlos**.

4.2 Cuándo conviene un proceso y cuándo un hilo

Pensemos en un videojuego. Los enemigos, la física, el sonido y la lógica de la partida comparten el mapa, la puntuación y las vidas: **son hilos del mismo proceso**, porque

necesitan ver los mismos datos al instante. En cambio, el navegador que tienes abierto detrás es **otro proceso**: no comparte nada con el juego, ni debe hacerlo.

	Hilos	Procesos
Memoria	Compartida	Separada
Si uno falla	Cae todo el proceso	Los demás siguen funcionando
Compartir datos	Inmediato	Hay que comunicarlos

De aquí se obtiene una regla práctica:

- Tareas que trabajan **sobre los mismos datos** → **hilos**.
- Tareas **independientes**, o que deben **aislarse** por seguridad o robustez → **procesos**.



Idea clave

Chrome abre un proceso por cada pestaña precisamente para que **una página que falla no cierre el navegador entero**.

Ejemplo de uso de hilos: hilos en un videojuego

Un **videojuego** es un ejemplo ideal para entender los hilos:

- Cada personaje o elemento que se mueve en el juego puede ser gestionado por un hilo distinto.
- Mientras el jugador controla un personaje, otros elementos como enemigos, obstáculos o efectos especiales están siendo gestionados por otros hilos, pero todos pertenecen al mismo proceso (el juego en sí).

En este contexto, los hilos comparten información del juego, como el **número de vidas** restantes, la **puntuación**, o la **posición** de cada personaje en el mapa. Cada hilo sigue un comportamiento definido, pero todos trabajan en conjunto bajo el mismo espacio de memoria del juego.



Ya sabemos qué es un proceso y cómo lo gestiona el sistema operativo. A partir de aquí, vamos a crearlos y controlarlos desde Java.

5 Creación y gestión de procesos desde Java

Hasta ahora hemos gestionado los procesos con las herramientas del sistema. En este apartado **los lanzará nuestro propio código**: crearemos procesos hijos, nos comunicaremos con ellos, esperaremos a que terminen y, si hace falta, los terminaremos.

5.1 Creación de procesos

5.1.1 Las clases Process y ProcessBuilder

Java proporciona dos clases, del paquete `java.lang`, con papeles distintos:

- **ProcessBuilder**: configura y lanza el proceso. Es el **plano** del proceso.
- **Process**: representa el proceso ya en marcha. Es el **proceso vivo**.

El esquema mínimo para lanzar un proceso es:

```
ProcessBuilder pb = new ProcessBuilder("tasklist");  
Process p = pb.start();
```

El método `start()` puede lanzar una `IOException`, por ejemplo si el ejecutable no existe, así que hay que declararla o capturarla.



Atención

En Internet encontrarás ejemplos con `Runtime.getRuntime().exec("...")`. **Este método está deprecado desde Java 18: no lo utilices.** En el apartado 5.1.2 veremos por qué.

La clase ProcessBuilder

Un objeto `ProcessBuilder` guarda la configuración del proceso, pero todavía no lanza nada. Antes de lanzarlo se puede configurar:

- El **comando** y sus **argumentos**.
- El **directorio de trabajo**.
- Las **variables de entorno**.
- A dónde van la **entrada**, la **salida** y los **errores** del proceso (apartado 5.2).

Los métodos de configuración devuelven el propio `ProcessBuilder`, por lo que se pueden **encadenar**:

```
Process p = new ProcessBuilder("tasklist", "/F0", "CSV")  
    .directory(new File("C:\\practicas"))  
    .start();
```

Además, un mismo `ProcessBuilder` puede lanzar varios procesos: cada llamada a `start()` crea un proceso nuevo e independiente.



Idea clave

Igual que un ejecutable puede dar lugar a muchos procesos, **un `ProcessBuilder` puede lanzar muchos hijos.**

La clase `Process`

El objeto `Process` lo devuelve `start()` y permite controlar el proceso hijo. Para consultar su información dispone de:

- `pid()`: el identificador del proceso, el mismo que muestra el Administrador de tareas.
- `isAlive()`: indica si el proceso sigue en ejecución.
- `info()`: información adicional, como el comando, el usuario que lo lanzó y la hora de inicio.

Además, `Process` permite **comunicarse con el hijo** a través de sus flujos (apartado 5.2) y **esperarlo y recoger su resultado** (apartado 5.3).



Atención

Un objeto `Process` no se crea con `new`. `Process` es una clase abstracta: el objeto concreto lo fabrica `start()` y depende del sistema operativo.

5.1.2 Argumentos, directorio de trabajo y entorno

Argumentos

Cada argumento del comando se pasa por separado, como un elemento propio. Comparemos dos formas de ejecutar `tasklist /FI "IMAGENAME eq notepad.exe"`:

Incorrecto: se pasa todo el comando en una sola cadena.

```
new ProcessBuilder("tasklist /FI \"IMAGENAME eq notepad.exe\");
```

Java busca un ejecutable cuyo nombre sea literalmente esa cadena completa. Como no existe, `start()` lanza una `IOException`.

Correcto: tres elementos, el programa y sus dos argumentos.

```
new ProcessBuilder("tasklist", "/FI", "IMAGENAME eq notepad.exe");
```

El tercer argumento contiene espacios y no hay ningún problema: como Java no tiene que trocear la cadena, **no hacen falta comillas de escape ni hay ambigüedad posible.**



Atención

Esta es precisamente la razón por la que se deprecó `Runtime.exec(String)`: recibía el comando en una sola cadena y tenía que adivinar dónde empezaba cada argumento.

Algunas órdenes, como `dir` o `echo`, no son ejecutables, sino **comandos internos de la consola**. Para lanzarlas hay que ejecutar `cmd` con la opción `/c`, que ejecuta la orden indicada y termina:

```
new ProcessBuilder("cmd", "/c", "dir");
```

Directorio de trabajo

El directorio de trabajo es **la carpeta desde la que se ejecuta el proceso hijo**. Por defecto, el hijo hereda el directorio del proceso padre, es decir, la carpeta desde la que se lanzó el programa Java. En IntelliJ, ese directorio es la raíz del proyecto.

Se cambia con el método `directory()`:

```
ProcessBuilder pb = new ProcessBuilder("cmd", "/c", "dir");  
pb.directory(new File("C:\\practicas"));  
Process p = pb.start();
```

Es importante tenerlo presente porque **todas las rutas relativas que use el hijo se resuelven desde ese directorio**. Si el hijo escribe en `salida.txt`, el archivo aparecerá en esa carpeta, no en la de tu proyecto. Además, la carpeta debe existir; si no, `start()` lanza una `IOException`.



Atención

Fallo típico: «el programa no genera el archivo». Sí lo genera, pero **en otra carpeta**. Comprueba el directorio de trabajo.

Variables de entorno

Las **variables de entorno** son datos de configuración que el sistema pasa a cada proceso, como `PATH`, `TEMP` o `USERNAME`. Son la forma estándar de configurar un programa sin modificar su código.

Se consultan y modifican con el método `environment()`, que devuelve un `Map<String, String>`:

```
ProcessBuilder pb = new ProcessBuilder("cmd", "/c", "echo %MOD0%");  
Map<String, String> entorno = pb.environment();  
entorno.put("MOD0", "PRUEBAS");  
Process p = pb.start();           // el hijo mostrara: PRUEBAS
```

Los cambios solo afectan al hijo: `environment()` devuelve una copia, así que ni el sistema ni tu programa Java se modifican.



Idea clave

Los argumentos y el entorno son **la primera forma de pasar información a un subprocesso**: se fijan antes de lanzarlo. En el apartado 5.2 veremos la segunda: enviarle datos mientras se ejecuta.

5.1.3 Lanzamiento y ciclo de vida del proceso hijo

Hasta la llamada a `start()` solo hay configuración. `start()` es el momento en que se crea el proceso real:

```
Process p = pb.start();
System.out.println("PID del hijo: " + p.pid());
System.out.println("Vivo: " + p.isAlive());
```

`start()` devuelve el control inmediatamente: el padre no se detiene. A partir de ese momento hay dos procesos ejecutándose a la vez, el padre y el hijo, cada uno a su ritmo.

El ciclo de vida del hijo es: **nace** con `start()` → **se ejecuta** de forma independiente → **termina** por su cuenta → **deja un código de salida**.



Atención

El padre puede terminar antes que el hijo, y el hijo seguirá ejecutándose. Para esperarlo y saber cómo terminó, se usa lo visto en el apartado 5.3; para leer lo que ha escrito, lo visto en el apartado 5.2.

Ejemplo: proceso padre e hijos

El siguiente programa lanza tres procesos hijos, muestra sus PID y permite comprobar con el Administrador de tareas o con `tasklist` que el padre y los hijos son procesos distintos:

```
import java.util.Scanner;

/**
 * Objetivo: ver quien es el proceso padre y quienes son los hijos.
 * Ejecutar desde IntelliJ y NO cerrar la ventana:
 * los hijos siguen vivos 20 segundos.
 */
public class PadreHijos {

    private static final int NUM_HIJOS = 3;

    public static void main(String[] args) throws Exception {
        // ===== 1. EL PADRE =====
        // Este programa Java, al ejecutarse, ya ES un proceso: el proceso PADRE.
        // ProcessHandle.current() representa a nuestro propio proceso.
        long pidPadre = ProcessHandle.current().pid();
        System.out.println("PADRE (este programa Java) -> PID " + pidPadre);

        long inicio = System.currentTimeMillis();

        // ===== 2. LOS HIJOS =====
        for (int i = 1; i <= NUM_HIJOS; i++) {
            // El comando va troceado: cada argumento es un elemento.
            // El hijo escribe un mensaje y despues se queda 20 segundos ocupado
            // (el ping a 127.0.0.1 es solo una forma de hacerle esperar).
        }
    }
}
```

```

        ProcessBuilder pb = new ProcessBuilder(
            "cmd", "/c",
            "echo      - Soy el HIJO " + i
            + " y escribo en la consola de mi padre"
            + " & ping -n 21 127.0.0.1 > nul");

        // inheritIO(): el hijo usa la MISMA consola que el padre.
        // Sin esta linea no veriamos nada de lo que escribe el hijo.
        pb.inheritIO();

        Process hijo = pb.start();    // <--- AQUI nace el proceso hijo
        System.out.println("HIJO " + i + " lanzado -> PID " + hijo.pid()
            + " (vivo: " + hijo.isAlive() + ")");
    }

    long tardado = System.currentTimeMillis() - inicio;

    // ===== 3. LA FAMILIA =====
    System.out.println("\nEl padre ha tardado " + tardado
        + " ms en lanzar a los " + NUM_HIJOS + " hijos.");
    System.out.println("No los ha esperado: start() no bloquea al padre.\n");

    System.out.println("Hijos que el padre reconoce ahora mismo:");
    ProcessHandle.current()
        .children()
        .forEach(h -> System.out.println("    - PID " + h.pid()));

    // ===== 4. COMPROBACION =====
    System.out.println("\nAHORA: busca los PID en el Administrador de tareas");
    System.out.println("o escribe en una consola:");
    System.out.println("    tasklist /FI \"PID eq \" + pidPadre + "\"");
    System.out.println("y busca tambien los PID de los hijos.\n");

    System.out.println("Pulsa INTRO para que el padre termine...");
    new Scanner(System.in).nextLine();
    System.out.println("PADRE terminado. Los hijos pueden seguir vivos.");
}
}

```

Al ejecutarlo observarás que el padre tarda muy pocos milisegundos en lanzar a los tres hijos y que los mensajes de los hijos aparecen mezclados con los del padre, porque todos se ejecutan a la vez.

5.2 Comunicación entre procesos

5.2.1 Entrada, salida y error estándar

Cada proceso tiene su propia memoria y no puede leer la de otro: el sistema operativo lo impide por seguridad. Para intercambiar datos, los procesos necesitan **canales de comunicación**.

Todo proceso nace con tres canales abiertos:

- **Entrada estándar (*stdin*):** de donde el proceso lee los datos que necesita. Normalmente es el teclado, pero puede ser un fichero u otro proceso.
- **Salida estándar (*stdout*):** donde el proceso escribe sus resultados. Normalmente es la pantalla, pero puede ser un fichero u otro proceso que necesite esos datos.
- **Salida de error (*stderr*):** donde el proceso escribe sus mensajes de error. Normalmente también es la pantalla, pero puede dirigirse a otro destino, como un fichero de errores.

La salida de error va separada de la salida estándar para **poder distinguir los resultados de los mensajes de error**: por ejemplo, guardar los resultados en un fichero y mostrar los errores en pantalla. En Java, `System.out` escribe en la salida estándar y `System.err` en la salida de error.



Idea clave

Estos tres canales son el **mecanismo de comunicación entre procesos más antiguo y más utilizado**.

5.2.2 Los flujos del hijo vistos desde el padre

El **proceso hijo que lanza Java no tiene consola propia**. Sus tres canales quedan conectados al padre mediante **tuberías (pipes)**: el padre es su único interlocutor. Para acceder a ellos, `Process` ofrece tres métodos:

Método de Process	Conecta con	Se utiliza para
<code>getOutputStream()</code>	stdin del hijo	Escribir datos al hijo
<code>getInputStream()</code>	stdout del hijo	Leer lo que produce el hijo
<code>getErrorStream()</code>	stderr del hijo	Leer los errores del hijo



Idea clave

Los nombres de los métodos describen **los flujos del padre**, no los del hijo: **la salida del padre es la entrada del hijo**, y al revés.

Leer lo que escribe el hijo

El siguiente programa ejecuta `tasklist` y muestra su salida línea a línea:

```
import java.io.BufferedReader;
import java.io.InputStreamReader;

public class LeerSalidaDelHijo {

    public static void main(String[] args) throws Exception {
        Process p = new ProcessBuilder("tasklist").start();

        try (BufferedReader br = new BufferedReader(
            new InputStreamReader(p.getInputStream()))) {
            String linea;
            while ((linea = br.readLine()) != null) {
                System.out.println("[hijo] " + linea);
            }
        }
    }
}
```

La lectura se hace en tres capas:

- `getInputStream()` proporciona **bytes**.

- `InputStreamReader` convierte esos bytes en **caracteres**.
- `BufferedReader` permite leer **líneas completas** con `readLine()`.

`readLine()` devuelve `null` cuando el hijo cierra su salida, normalmente al terminar. **Ese `null` es la señal de fin.**



Atención

La tubería tiene una capacidad limitada. **Si el hijo escribe mucho y nadie lee, la tubería se llena y el hijo se queda bloqueado para siempre.** Leer la salida no es opcional.



Atención

La consola de Windows no utiliza UTF-8, así que las tildes y eñes de la salida de comandos como `tasklist` pueden verse mal. Se corrige indicando la codificación de la consola al leer: `new InputStreamReader(p.getInputStream(), "CP850")`.

Leer los errores

Si solo se lee `getInputStream()`, **los mensajes de error se quedan en el otro canal y el hijo puede fallar sin que nos enteremos.** Hay dos opciones:

Opción A: unir los dos canales antes de lanzar el proceso.

```
ProcessBuilder pb = new ProcessBuilder("cmd", "/c", "dir zzz");
pb.redirectErrorStream(true);      // stderr pasa a stdout
Process p = pb.start();           // ahora se lee todo junto
```

Con `redirectErrorStream(true)` no se pueden distinguir los errores de los resultados, pero se evita el bloqueo. **Para empezar, es la opción recomendable.**

Opción B: leer los dos canales por separado, usando `getErrorStream()` con otro `BufferedReader`. Así se distinguen, pero hay que leer los dos a la vez: mientras se espera a uno, el otro puede llenarse y bloquear al hijo.

Escribir en la entrada del hijo

Escribiendo en la entrada estándar del hijo **le pasamos información mientras se ejecuta.** En este ejemplo usamos `sort`, un comando de Windows que lee líneas de su entrada estándar y las devuelve ordenadas:

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.PrintWriter;

/**
 * Enviamos datos a un proceso hijo ('sort' de Windows) y leemos su respuesta.
 */
public class HablarConElHijo {

    public static void main(String[] args) throws Exception {
        Process hijo = new ProcessBuilder("sort").start();

        String[] palabras = {"pera", "manzana", "kiwi", "cereza"};

        // getOutputStream() es NUESTRA salida... y la ENTRADA del hijo.
```

```
// Al cerrarse (fin del try), el hijo sabe que no hay mas datos.
try (PrintWriter haciaElHijo = new PrintWriter(hijo.getOutputStream())) {
    for (String palabra : palabras) {
        System.out.println("    -> " + palabra);
        haciaElHijo.println(palabra);
    }
}

// getInputStream() es NUESTRA entrada... y la SALIDA del hijo.
try (BufferedReader desdeElHijo = new BufferedReader(
    new InputStreamReader(hijo.getInputStream()))) {
    String linea;
    while ((linea = desdeElHijo.readLine()) != null) {
        System.out.println("    <- " + linea);
    }
}
}
```

Cerrar el flujo de escritura es imprescindible. Mientras siga abierto, el hijo considera que faltan datos por llegar y sigue esperando. Al cerrarlo, le indicamos que no hay más datos y el hijo empieza a trabajar.



Idea clave

Escribir, cerrar y después leer la salida: ese es el ciclo completo de comunicación con un subprocesso.

5.2.3 Redirecciones y encadenamiento de procesos

A veces no necesitamos procesar la salida del hijo desde Java: solo queremos que aparezca en pantalla o que se guarde en un fichero. Para eso se usan las **redirecciones**, que se configuran en el **ProcessBuilder** antes de lanzar el proceso.

Heredar la consola del padre

```
pb.inheritIO();    // el hijo escribe directamente en la consola del padre
```

Redirigir a ficheros

```
pb.redirectOutput(new File("salida.txt")); // stdout del hijo -> salida.txt
pb.redirectError(new File("errores.txt")); // stderr del hijo -> errores.txt
pb.redirectInput(new File("datos.txt"));   // stdin del hijo <- datos.txt
```

Por defecto, el fichero de salida se sobrescribe. Para **añadir al final** en lugar de sobrescribir:

```
pb.redirectOutput(ProcessBuilder.Redirect.appendTo(new File("log.txt")));
```



Idea clave

Con redirecciones no hay tubería que se llene, así que el bloqueo desaparece. Es la opción más sencilla cuando no necesitas los datos en Java.

Encadenar procesos

Se puede conectar **la salida de un proceso directamente con la entrada del siguiente**, igual que hace la consola con el símbolo `|`. Para ello, desde Java 9 existe el método `ProcessBuilder.startPipeline()`:

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.util.List;

public class EncadenaProcesos {

    public static void main(String[] args) throws Exception {
        List<Process> cadena = ProcessBuilder.startPipeline(List.of(
            new ProcessBuilder("tasklist"),
            new ProcessBuilder("findstr", "chrome")));

        Process ultimo = cadena.get(cadena.size() - 1);

        // Solo se lee la salida del ultimo proceso de la cadena
        try (BufferedReader br = new BufferedReader(
            new InputStreamReader(ultimo.getInputStream()))) {
            String linea;
            while ((linea = br.readLine()) != null) {
                System.out.println(linea);
            }
        }
    }
}
```

`startPipeline()` **lanza todos los procesos a la vez** y conecta la salida de cada uno con la entrada del siguiente. **Solo hay que leer la salida del último**. Este ejemplo equivale a ejecutar en consola `tasklist | findstr chrome`.

Ya sabemos comunicarnos con el hijo. Falta lo más importante: esperararlo y saber si ha terminado bien o mal.

5.3 Sincronización entre procesos

Como vimos, `start()` **devuelve el control al instante** y a partir de ahí padre e hijo se ejecutan a la vez. Esto provoca situaciones como la siguiente:

```
public class ProblemaSincronizacion {

    public static void main(String[] args) throws Exception {
        Process p = new ProcessBuilder("cmd", "/c", "ping -n 6 127.0.0.1").start();
        System.out.println("Ya he terminado");    // se imprime de inmediato
    }
}
```

El mensaje «Ya he terminado» aparece cuando el hijo apenas ha empezado. Esto es un problema cuando:

- El hijo genera un archivo que el padre necesita leer después.
- El resultado de un proceso es la entrada del siguiente.
- El padre necesita saber si el trabajo del hijo salió bien.



Idea clave

En esta unidad de trabajo, **sincronizar significa esperar al hijo y recoger su resultado**. En la unidad de trabajo 2, esta palabra tendrá otro significado.

5.3.1 Espera del proceso hijo: `waitFor()`

El método `waitFor()` **detiene al padre hasta que el hijo termina** y devuelve su código de salida:

```
public class EsperaAlHijo {

    public static void main(String[] args) throws Exception {
        Process p = new ProcessBuilder("cmd", "/c", "ping -n 6 127.0.0.1").start();
        int codigo = p.waitFor(); // el padre se queda aqui parado
        System.out.println("El hijo ha terminado con codigo " + codigo);
    }
}
```

Mientras espera, **el padre pasa al estado Bloqueado** que estudiamos en el apartado 2.1. No consume procesador: espera un evento externo, igual que si esperara al disco.



Atención

`waitFor()` puede lanzar una `InterruptedException`. Hay que declararla o capturarla.

Esperar con límite de tiempo

Si el hijo se queda colgado, `waitFor()` bloquearía al padre para siempre. Para evitarlo, existe una versión con límite de tiempo, que usa la clase `java.util.concurrent.TimeUnit`:

```
boolean aTiempo = p.waitFor(10, TimeUnit.SECONDS);
```

Devuelve `true` si el hijo terminó dentro del plazo y `false` si el plazo se agotó y el hijo sigue vivo. **Así, un hijo colgado no deja colgado también al padre.**

5.3.2 Códigos de salida

Todo proceso devuelve un número entero al terminar: su código de salida. Es la forma en que el hijo informa al padre de cómo le ha ido. El convenio es universal:

- **0:** todo ha ido bien.
- **Distinto de 0:** algo ha fallado. Cada programa define el significado de cada número.

Hay dos formas de obtener el código de salida:

- `waitFor()`: espera a que el hijo termine y devuelve el código.
- `exitValue()`: devuelve el código sin esperar, pero **lanza una `IllegalThreadStateException` si el hijo sigue vivo.**

```
if (p.waitFor() == 0) {  
    System.out.println("Correcto");  
} else {  
    System.out.println("El hijo ha fallado");  
}
```

Por ejemplo, `cmd /c dir carpeta_inexistente` termina con código 1, porque no encuentra la carpeta.

Un programa Java también devuelve su código de salida con `System.exit()`. Por ejemplo, `System.exit(3)`; termina la máquina virtual con código 3. Así, un programa Java puede informar de su resultado a otro programa que lo haya lanzado.



Idea clave

En la consola de Windows puedes comprobar el código de salida del último comando ejecutado con `echo %ERRORLEVEL%`.

5.3.3 Espera asíncrona: `onExit()`

A veces no queremos detener al padre, sino que **nos avise cuando el hijo termine**. Para eso se usa `onExit()`.

`onExit()` devuelve un objeto `CompletableFuture<Process>`: una promesa de que ese proceso terminará. Con su método `thenAccept()` indicamos qué hay que hacer cuando ocurra.

```
import java.util.concurrent.CompletableFuture;  
  
public class AvisoAlTerminar {  
  
    public static void main(String[] args) throws Exception {  
        Process p = new ProcessBuilder("cmd", "/c", "ping -n 6 127.0.0.1").start();  
  
        // Registramos lo que hay que hacer cuando el hijo termine  
        CompletableFuture<Void> aviso = p.onExit().thenAccept(proceso ->  
            System.out.println("Aviso: el hijo " + proceso.pid()  
                + " ha terminado"));  
  
        System.out.println("El padre sigue trabajando mientras tanto...");  
  
        // Solo para la prueba: evitamos que el padre termine antes que el aviso  
        aviso.join();  
    }  
}
```

La diferencia con `waitFor()` es clara:

- `waitFor()` **bloquea al padre** hasta que el hijo termina.
- `onExit()` **no bloquea al padre**: registra el aviso y el padre continúa.



Atención

Si el padre termina antes que el hijo, el aviso no llega a ejecutarse. Por eso, en el ejemplo, la última línea (`aviso.join()`) espera a que el aviso se haya mostrado.

5.3.4 Consulta y terminación: `ProcessHandle` y `destroy()`

`ProcessHandle`: la vista del sistema

`Process` controla un proceso que hemos creado nosotros. `ProcessHandle` permite **consultar cualquier proceso del sistema**.

Para obtener el proceso actual, es decir, nuestro propio programa:

```
ProcessHandle yo = ProcessHandle.current();
System.out.println("Mi PID: " + yo.pid());
```

Para buscar cualquier proceso a partir de su PID:

```
Optional<ProcessHandle> ph = ProcessHandle.of(4820);
```

Devuelve un `Optional` porque **ese PID puede no existir**. A través de `info()` se obtiene información del proceso:

- `info().command()`: la ruta del ejecutable.
- `info().startInstant()`: el momento en que arrancó.
- `info().user()`: el usuario que lo lanzó.

Es la versión programable de lo que hacíamos manualmente con `tasklist` en el apartado 3.

Hijos, nietos y descendientes

El sistema organiza los procesos en un **árbol de padres e hijos**. `ProcessHandle` (y también `Process`) ofrece dos métodos para recorrerlo:

- `children()`: **solo los hijos directos**.
- `descendants()`: **hijos, nietos y toda la rama** que cuelga del proceso.

```
ProcessHandle.current().children()
    .forEach(h -> System.out.println("Hijo: " + h.pid()));

ProcessHandle.current().descendants()
    .forEach(h -> System.out.println("Descendiente: " + h.pid()));
```

Un caso real: si lanzamos `cmd /c ping`, nuestro hijo es `cmd` y `ping` es nuestro **nieto**. `ping` no aparece en `children()`, pero sí en `descendants()`.

Terminar un proceso desde Java

Existen dos métodos, equivalentes a `taskkill /PID` y `taskkill /PID /F` del apartado 3.2:

```
p.destroy();           // solicita la terminacion
p.destroyForcibly();    // terminacion inmediata
```



Atención

En **Windows**, `destroy()` termina el proceso de forma inmediata, igual que `destroyForcibly()`: el proceso no tiene ocasión de cerrar ordenadamente ni de guardar nada. El método `p.supportsNormalTermination()` devuelve `false` en Windows.

Además, **terminar un proceso no termina a sus hijos**: si se destruye `cmd`, el `ping` nieto sigue vivo.

Por eso, la forma segura de terminar un hijo y toda su rama es:

```
p.descendants().forEach(ProcessHandle::destroyForcibly); // primero los nietos
p.destroy();
if (!p.waitFor(5, TimeUnit.SECONDS)) {
    p.destroyForcibly();
}
```

La llamada a `waitFor()` con límite de tiempo **comprueba que el proceso ha terminado de verdad** antes de continuar.

Ya sabemos crear procesos, comunicarnos con ellos, esperarlos y terminarlos. En el siguiente apartado los usaremos para repartir trabajo.

6 Aplicaciones multiproceso en paralelo

Con todo lo visto en el apartado 5 ya podemos construir aplicaciones que **repartan un trabajo entre varios procesos** para terminarlo antes.

6.1 Reparto de una tarea entre varios subprocessos

La idea es **dividir una tarea grande en partes independientes que se ejecutan a la vez**. Siempre se sigue el mismo patrón en tres pasos:

1. **Dividir**: partir el trabajo en trozos que no dependan unos de otros.
2. **Lanzar**: crear un subprocesso por trozo, todos a la vez.
3. **Recoger**: esperar a que terminen todos y juntar los resultados.

Ejemplos reales de este patrón son el **procesamiento de imágenes**, donde cada subprocesso aplica un filtro a una parte de la imagen, y las **simulaciones científicas**, donde cada subprocesso calcula una parte del modelo.



Imagen 10.- Dividir, lanzar y recoger (unir) resultados



Idea clave

La clave está en el primer paso. Si los trozos dependen unos de otros, no se pueden ejecutar a la vez y no hay paralelismo posible.

El error más frecuente: lanzar y esperar en el mismo bucle

```
for (int i = 0; i < 4; i++) {
    Process p = pb.start();
    p.waitFor();          // MAL: espera a cada uno antes de lanzar el siguiente
}
```

En este código **no hay paralelismo**: cada proceso se lanza cuando ha terminado el anterior, así que los cuatro se ejecutan uno detrás de otro. La forma correcta es **usar dos bucles separados**: primero se lanzan todos y después se espera a todos.

```
List<Process> procesos = new ArrayList<>();

for (int i = 0; i < 4; i++) {                // 1) lanzar todos
    procesos.add(pb.start());
}
for (Process p : procesos) {                 // 2) esperarlos despues
    p.waitFor();
}
```



Atención

Lanzar y esperar tienen que ir en bucles distintos. Es el fallo más frecuente en este tipo de programas.

Recoger los resultados comprobándolos

No basta con esperar a los subprocesos: **hay que comprobar que cada parte ha terminado bien**, revisando su código de salida.

Además, como cada proceso tiene su propia memoria, **no hay variables compartidas** en las que dejar los resultados. La solución habitual es que **cada subprocesso escriba en su propio archivo**, redirigiendo su salida con `redirectOutput()`, y que el padre junte después esos archivos:

```
List<Process> procesos = new ArrayList<>();

for (int i = 1; i <= 4; i++) {
    ProcessBuilder pb = new ProcessBuilder(
        "cmd", "/c", "echo Resultado de la parte " + i);
    pb.redirectOutput(new File("parte" + i + ".txt")); // un archivo por hijo
    procesos.add(pb.start());
}

int fallos = 0;
for (Process p : procesos) {
    if (p.waitFor() != 0) {
        fallos++;
    }
}
System.out.println("Subprocesos fallidos: " + fallos);
```



¿Por qué no escriben todos en un mismo archivo? Porque **varios procesos escribiendo a la vez en el mismo archivo producen líneas mezcladas o datos perdidos**. Por eso se crea un archivo por subprocesso y se unen al final.

Idea clave

Comunicar datos entre procesos tiene un coste. Por eso **se reparten trabajos grandes, no tareas diminutas**.

6.2 Medición frente a la ejecución secuencial

Para saber si la versión en paralelo es realmente más rápida, **hay que medir**. El tiempo se mide con `System.nanoTime()`:

```
long inicio = System.nanoTime();
// ... lanzar y esperar a todos ...
long ms = (System.nanoTime() - inicio) / 1_000_000;
System.out.println("Tiempo: " + ms + " ms");
```



Atención

`System.nanoTime()` sirve para **medir intervalos de tiempo**. `System.currentTimeMillis()` sirve para **saber la hora**. No los confundas.

Cómo hacer una comparación válida

- **Mismo trabajo, misma máquina y mismos datos** en las dos versiones.

- Primero se ejecuta la versión secuencial y después la paralela.
- Se repite **tres veces** cada una y se toma la **mediana** (el valor central).

La mejora se expresa con la **aceleración**:

$$\text{Aceleración} = \text{tiempo secuencial} \div \text{tiempo paralelo}$$

Si la aceleración es 2,5, la versión paralela es dos veces y media más rápida que la secuencial.

Ejemplo: secuencial frente a paralelo

El siguiente programa ejecuta cuatro tareas de 5 segundos, primero de forma secuencial y después en paralelo, y calcula la aceleración:

```
import java.util.ArrayList;
import java.util.List;

/**
 * La misma carga de trabajo, hecha de dos formas: secuencial y paralela.
 * Cada tarea son 5 segundos de espera (un ping que no muestra nada).
 */
public class SecuencialVsParalelo {

    private static final int N = 4;

    // Una tarea = 5 segundos ocupado
    private static ProcessBuilder tarea() {
        return new ProcessBuilder("cmd", "/c", "ping -n 6 127.0.0.1 > nul");
    }

    public static void main(String[] args) throws Exception {
        // ----- SECUENCIAL: lanzar y esperar, uno a uno -----
        long t0 = System.nanoTime();
        for (int i = 0; i < N; i++) {
            tarea().start().waitFor(); // espera antes del siguiente
        }
        long secuencial = (System.nanoTime() - t0) / 1_000_000;

        // ----- PARALELO: lanzar todos, esperar despues -----
        t0 = System.nanoTime();
        List<Process> procesos = new ArrayList<>();
        for (int i = 0; i < N; i++) {
            procesos.add(tarea().start()); // bucle 1: solo lanzar
        }
        for (Process p : procesos) {
            p.waitFor(); // bucle 2: ahora si, esperar
        }
        long paralelo = (System.nanoTime() - t0) / 1_000_000;

        System.out.printf("Secuencial: %d ms\n", secuencial);
        System.out.printf("Paralelo: %d ms\n", paralelo);
        System.out.printf("Aceleracion: %.2f\n", (double) secuencial / paralelo);
    }
}
```

El resultado será aproximadamente 20 000 ms en secuencial, algo más de 5 000 ms en paralelo y una aceleración cercana a 4. Ten en cuenta que en este ejemplo cada tarea **espera** en lugar de calcular; con tareas que usan intensamente el procesador, la aceleración queda limitada por el número de núcleos.

¿Por qué la aceleración nunca es proporcional al número de núcleos?

- Crear cada proceso cuesta tiempo, y se crean varios.

- Siempre hay **una parte que no se puede repartir**: preparar los datos y juntar los resultados.
- **El disco y la memoria son compartidos**: varios procesos accediendo a la vez se estorban entre sí.

Por tanto, **el paralelismo solo compensa si cada trozo dura bastante más de lo que cuesta crear su proceso**. Para tareas de milisegundos, el paralelismo sale más caro que el propio trabajo. Y lanzar más procesos que núcleos no mejora el resultado: el planificador vuelve a repartir el procesador por turnos, como vimos en el apartado 2.



Idea clave

Primero mide, después decide. Paralelizar sin medir suele empeorar las cosas.

7 Documentación y depuración

Una aplicación multiproceso debe quedar **documentada**, para que otra persona entienda qué hace cada proceso, y **depurada**, para comprobar que funciona correctamente.

Documentación con Javadoc

Javadoc es la herramienta estándar de Java para documentar el código. Se basa en comentarios que empiezan por `/**` y se colocan justo antes de cada clase o método. Dentro del comentario se usan etiquetas:

- `@param`: describe cada parámetro del método.
- `@return`: describe el valor que devuelve.
- `@throws`: indica las excepciones que puede lanzar y cuándo.

```
/**
 * Lanza una tarea como subprocesso y espera a que termine.
 *
 * @param numParte numero de la parte que procesa el subprocesso
 * @param salida  archivo donde el subprocesso escribe su resultado
 * @return codigo de salida del subprocesso (0 si ha terminado correctamente)
 * @throws IOException      si no se puede lanzar el subprocesso
 * @throws InterruptedException si se interrumpe la espera
 */
public static int lanzarParte(int numParte, File salida)
    throws IOException, InterruptedException {
    ProcessBuilder pb = new ProcessBuilder("cmd", "/c", "echo Parte " + numParte);
    pb.redirectOutput(salida);
    return pb.start().waitFor();
}
```

En una aplicación multiproceso conviene documentar, además, **qué comando ejecuta cada subprocesso, dónde deja sus resultados y qué significa cada código de salida**.

A partir de estos comentarios, IntelliJ genera la documentación en formato HTML desde el menú **Tools > Generate JavaDoc...**, indicando la carpeta de destino. El resultado se consulta abriendo el archivo [index.html](#) en el navegador.



[Vídeo: generar la documentación con IntelliJ](#)



[Vídeo: generar la documentación con Eclipse](#)

Dificultades de la depuración con varios procesos

Depurar una aplicación multiproceso es más difícil que depurar un programa secuencial, por tres motivos:

- **El depurador de IntelliJ solo entra en el proceso padre.** Los puntos de ruptura (*breakpoints*) detienen al padre, pero el hijo es otro proceso, que puede ni siquiera estar escrito en Java.
- **Los errores del hijo pueden pasar inadvertidos:** si nadie lee su salida de error ni comprueba su código de salida, el hijo falla en silencio.
- **El orden de ejecución varía** de una ejecución a otra, porque padre e hijos se ejecutan a la vez.

Técnicas de depuración

1. **Probar primero el comando en la consola.** Antes de lanzarlo desde Java, ejecútalo a mano en `cmd` y comprueba que funciona.
2. **Mostrar el comando que se va a lanzar** con `pb.command()`, para verificar que los argumentos están bien separados.
3. **Comprobar siempre el código de salida** con `waitFor()`.
4. **No perder la salida de error:** leerla, unirla a la salida estándar con `redirectErrorStream(true)` o guardarla en un archivo con `redirectError()`.
5. **Dejar trazas** con el PID y el resultado de cada subproceso, para saber qué ha ocurrido y en qué orden.
6. **Usar puntos de ruptura en el padre** justo después de `start()` o de `waitFor()`, para inspeccionar el objeto `Process` (`pid()`, `isAlive()`, código de salida).

El siguiente fragmento aplica varias de estas técnicas:

```
ProcessBuilder pb = new ProcessBuilder("cmd", "/c", "dir", "carpeta_inexistente");
pb.redirectError(new File("errores.txt")); // guarda los errores del hijo

System.out.println("Lanzando: " + pb.command());
Process p = pb.start();
int codigo = p.waitFor();

if (codigo != 0) {
    System.err.println("El hijo " + p.pid() + " ha terminado con codigo " + codigo
        + ". Revisa errores.txt");
}
```



Idea clave

En una aplicación multiproceso, **el código de salida y la salida de error son la principal fuente de información** para localizar un fallo en un hijo.

Bibliografía

- [1] Cortijo Bon, Carlos A. (2021). *Programación de servicios y procesos*. Editorial Síntesis (España). ISBN: 978-84-1357-104-1.
- [2] Córcoles Tendero, José Eduardo. (2014). *Programación de servicios y procesos*. Editorial RA-MA (España). ISBN: 978-84-9964-391-5.
- [3] Paniagua Martín, Fernando. (2022). *Programación de servicios y procesos*. Editorial Paraninfo. ISBN: 978-84-1366-526-9.